

Unit 1 : Introduction to Linux Operating System

Linux is an open-source operating system that acts as the bridge between computer hardware and software. It manages hardware resources (like the CPU, memory, storage) and provides an environment where applications can run.

LINUX is an open-source operating system used extensively worldwide. It is free and its source code is available for everyone under **GNU(General Public Licence)**.

Linux as an Operating System provides a wide range of features that can be used by a wide range of systems such as servers and computers.

Android OS is also based on the Linux kernel that extends its usage for building embedded systems and automation systems.

What are the Features of Linux Operating Systems?

The following are the features of the Linux Operating system:

1. Free and Open-Source

Linux is completely free of cost, and expenses are never a hindrance to using it as an operating system.

Linux is open-source. This means that modification of code, analysis of codes, redistribution of codes, or selling copies of the enhanced codes can be done by anyone in the world provided they come under the same license where the license also costs no charge.

Linux operating system is released under the **GNU(General Public Licence)** and is now one of the largest open-source projects worldwide.

2. Extremely Flexible

Linux has incorporated itself into embedded products like watches, digital equipment and supercomputing servers.

There are no prerequisites for installing an entire *Linux suit*. It allows a user to install only the components that are required by the user.

3. Lightweight Infrastructure

Linux consumes lesser storage space, and its installation requires around 4GB to 8GB of disk space.

Memory footprint or the amount of memory (*RAM*) used by the software while running is also less and it is compatible with all kinds of file formats like *text files, audio files, video files, graphic formats*, etc.

4. Graphical User Interface (GUI)

Even though *Linux* works on using the command line interface but it can be converted to be used like windows having a *Graphical user interface*. This is done mostly by installing packages. The most common way of having a *GUI* on the *Linux* environment is to log in to **Ubuntu** server and install its desktop environment.

5. End-to-end encryption

Linux allows end-to-end encryption while accessing data thus storing public keys in the server. All data is password protected and provides authentication to users. It also allows many security features and provides file permissions, a secure shell, etc.

6. Portable Environment

Linux works on any kind of environment and doesn't depend on the device being high-ended (supercomputer/server) or low-ended(in router). A large number of users can simultaneously use it anytime, any place, and on multiple devices. It supports all kinds of hardware to work on.

Multiple distributions or enterprises are also supported by *Linux*.

Linux has its own repository/library for software that can be used to install the required packages.

7. Shell/ Command-line Interface

The *Linux* command line interpreter is known as **Shell** that provides an interface between the user and kernel which then executes programs known as commands.

Hence, *Linux* uses *Command-line interface* to carry out the execution of tasks which is comparatively more efficient to execute and takes lesser time. It also takes lesser space in the memory,

8. Customized keyboard

Linux is available in all languages which makes it acceptable worldwide. That is why language can be changed from the keyboard according to the requirements and preferences.

Users can also build their own customized layout for keyboards.

9. Frequent New Updates

Linux operating system provides a wide range of accessible software updates that can be deployed and used according to requirements.

They update more frequently thus giving users an option to choose the updates and install them as per their need.

10. Hierarchical File System

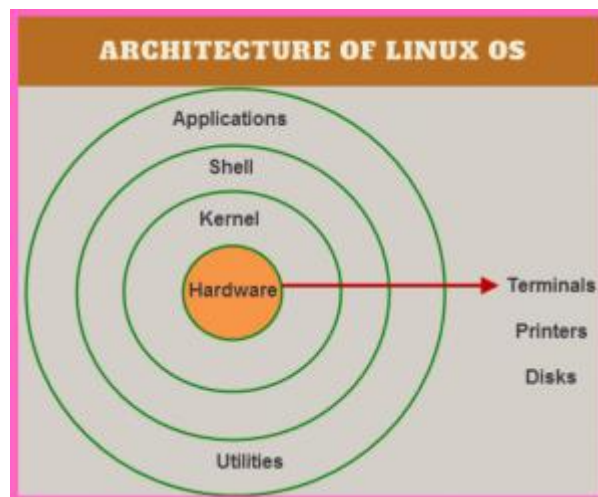
Linux comes with a well-defined file structure where user files are arranged in a definite directory structure. According to the type of files, the directories are categorised as *Binary directories, configuration directories, Data directories, memory directories, Usr(Unix System Resources), var(variable directory)* and *non standard directories*.=

11. Multi-user and Multi-programming

Linux allows multiple users to access the system resources at the same time and allows multiple applications to run at the same time

1.2 Components of Linux OS (Hardware, Kernel, Shell, GNU Utilities & Applications)/The architecture of the Linux operating system.

The structure of the Linux Operating System mainly has all these elements: Shell and System Utility, Hardware Layer, System Library, [Kernel](#).



Architecture of Linux OS

- **Shell and system utility in Linux operating system.**

A shell is a special software program that provides the software with an interface to access the operating system resources. Shell embraces user understandable human commands and transforms them into anything the kernel can recognize. This is an interpretation of the command language which executes read instructions from input devices including such a keyboard or documents. Once the user signs in or starts the terminal the shell gets underway.

- **The kernel in the Linux operating system.**

The kernel seems to be the origin of the Linux-based Shell. It provides virtual the device's common hardware components for providing its virtualized networks to each operation. This makes the whole process look like it is the only process that runs on the computer. This is also the kernel that is responsible for stopping and conflict resolution between procedures. Specific kernel forms are monolithic kernels, hybrid kernels, small kernels and kernels from Exo.

- **The hardware layer in the Linux operating system.**

That is the physical hardware that your device consists of; that includes items like your mouse and keyboard, the graphics chipset and display, and, if you do have one, your network interface card. Certain hardware parts that are not so evident are the Processor as well as the RAM inside your device.

- **The application and utility layer in the Linux operating system.**

A Linux-based [system](#) will typically arrive with such a set of established Unix-like utility services; these were all generally basic functions used in day-to-day operating system usage, and also specific devices and applications. This is usually software that was published and released under the open-source license by the GNU Project, therefore the software can be openly downloaded, updated, and redistributed to all. Some instances would have been the commands that allow users to share and modify files and folders, generate data as well as do jobs such as backup files.

1.3 Shell in Linux (Bash, Zsh, Dash – Features and Differences)

1. Bash (Bourne Again Shell)

- bash is the default shell on many Linux distributions and macOS.
- It is an enhanced version of the original Bourne Shell (sh), with additional features and improvements.

Features:

- Command-line editing.
- Job control.
- Shell functions and aliases.
- Brace expansion, command substitution, and more.

Syntax Example:

```
#!/bin/bash
```

```
echo "Hello, World!"
for i in {1..5}; do
    echo "Iteration $i"
done
```

2. Dash (Debian Almquist Shell)

- dash is a POSIX-compliant (programs and scripts will run in your OS) shell optimized for speed and small size.
- It is often used as `/bin/sh` on Debian-based systems.

Features:

- Minimalistic; focuses on POSIX compliance.
- Faster startup time compared to bash.

Syntax Example:

```
#!/bin/dash

echo "Hello, World!"
for i in $(seq 1 5); do
    echo "Iteration $i"
done
```

3. Zsh (Z Shell)

- zsh is known for its powerful features and flexibility.
- It combines features from bash, ksh (ksh combines features from sh (Bourne Shell for Unix shell script) and csh (C Shell like C language), and tcsh (enhanced version of the C Shell (csh)), and adds many unique capabilities.

Features:

- Programmable command-line completion.
- Loadable modules.
- Spelling correction.
- Theme and plugin support.

Syntax Example:

```
#!/bin/zsh

echo "Hello, World!"
for i in {1..5}; do
```

```
    echo "Iteration $i"
done
```

1.4 Introduction to Files and File Types in Linux (text, binary, special files)

In Linux, most of the operations are performed on files. And to handle these files Linux has directories also known as folders which are maintained in a tree-like structure.

In Linux, there are various types of files, which can be classified based on their attributes, such as permissions, ownership, and contents. Here are some common types of files:

File Symbol	File Type
-	Regular File
d	Directory
l	Link
c	Device File
s	Socket
p	FIFO or Named Pipe
b	Block Device

Linux File System

1. Regular Files:

- **Text files:** Files containing human-readable text (e.g., .txt, source code files).
- **Binary files:** Files containing compiled code, programs, or any non-readable data (e.g., executable files).
- **Image/Multimedia files:** Files containing media such as images, videos, or music (e.g., .jpg, .mp4).

2. Directory Files:

Directories (or folders) are special files that contain references to other files or directories. It is **denoted by d** at the beginning of the permission string in `ls -l`. for example : `/home/user,/bin,/dev`

3. Symbolic Link (symlink):

A symlink is a file that **special type of file** point to another file or directory. It acts as a **shortcut to another file**. It stores the **path to the target file**. It does not contain actual data in file.

We can create link file using
`ln -s p1.sh p2.sh`

4. Character Device Files:

These are special files that represent devices capable of sending and receiving data character by character, such as keyboards or serial ports.

These files are present in /dev folder

File that reads/writes data in character by character. Denoted by **c** at the beginning of the file type in `ls -l`.

Example: `ls -l /dev/input/mouse2`

Output: `crw-r--r-- 1 root root 13, 34 Jun 30 /dev/input/mouse2`

We can create manually using **mknod** command.

Example: — `/dev/input/mouse2`

5. Block Device Files:

Block device files represent devices that transfer data in blocks, like hard drives or USB drives. *A file that refers to a device.* Find under `/dev/`

Example:

Hard drives (`/dev/sda`)

USB drives (`/dev/sdb`)

CD/DVD drives (`/dev/sr0`)

Partitions (`/dev/sda1`, `/dev/sdb2`)

Denoted by **b** in the first character when listed with `ls -l`.

Example: `ls -l /dev/sda1`

Output: `brw-rw---- 1 root disk 8, 1 Jun 30 /dev/sda1`

We can create manually using **mknod** command.

6. FIFO (Named Pipes):

FIFO files are used for inter-process communication (IPC). They allow processes to communicate with each other by reading and writing data in a first-in, first-out sequence.

Sends data from one process to another so that the receiving process reads the data first-in-first-out manner.

Example: Ls|wc>p1

Cat p1

7. Socket Files:

Socket files are **special file types** used for **inter-process communication (IPC)**, especially for **network communication** between processes on the same system or across different systems.

- They allow bidirectional communication between client and server programs.
- Used in **client-server applications**.
- Support both **local (UNIX) domain** and **network (TCP/IP)** communication.
- Provide **fast and secure communication** within the system.
- “s” at the start indicates a **socket file** when ls-l run

Example: ls -l /run/chrony/chronyd.sock

Output: srwxr-xr-x 1 chrony chrony 0 Jun 30 /run/chrony/chronyd.sock

1.5 Linux Directory Structure and File System Hierarchy Standard (FHS)

Linux Directory Structure

In Linux, everything is treated as a file even if it is a normal file, a directory, or even a device such as a printer or keyboard. All the directories and files are stored under one root directory which is represented by a forward slash /.

The Linux directory layout follows the **Filesystem Hierarchy Standard (FHS)**. This standard defines how directories are organized and what types of files should be stored in each.

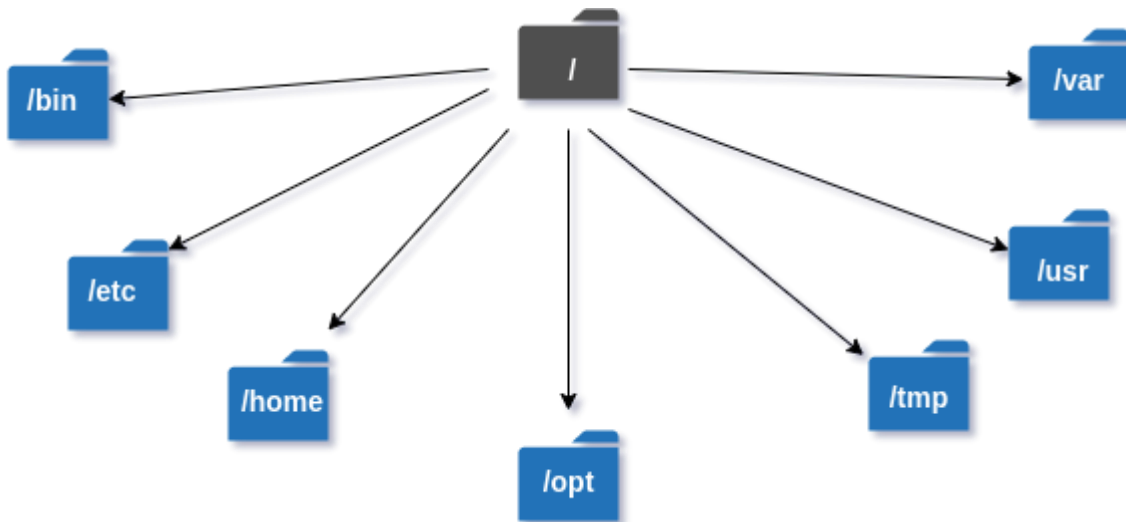
Types of files in the Linux system

Linux categorizes files into different types based on their purpose, such as general data storage, system structure, or device representation.

1. **General Files** - It is also called ordinary files. It may be an image, video, program, or simple text file. These types of files can be in ASCII or Binary format. It is the most commonly used file in the Linux system.
2. **Directory Files** - These types of files are a warehouse for other file types. It may be a directory file within a directory (subdirectory).
3. **Device Files** - In a Windows-like operating system, devices like CD-ROM and hard drives are represented as drive letters like F: G: H, whereas in the

Linux system, devices are represented as files. As for example, `/dev/sda1`, `/dev/sda2`, and so on.

We know that in a Windows-like operating system, files are stored in different folders on different data drives like C: D: E:, whereas in the Linux/Unix operating system, files are stored in a tree-like structure starting with the root directory, as shown in the diagram below.



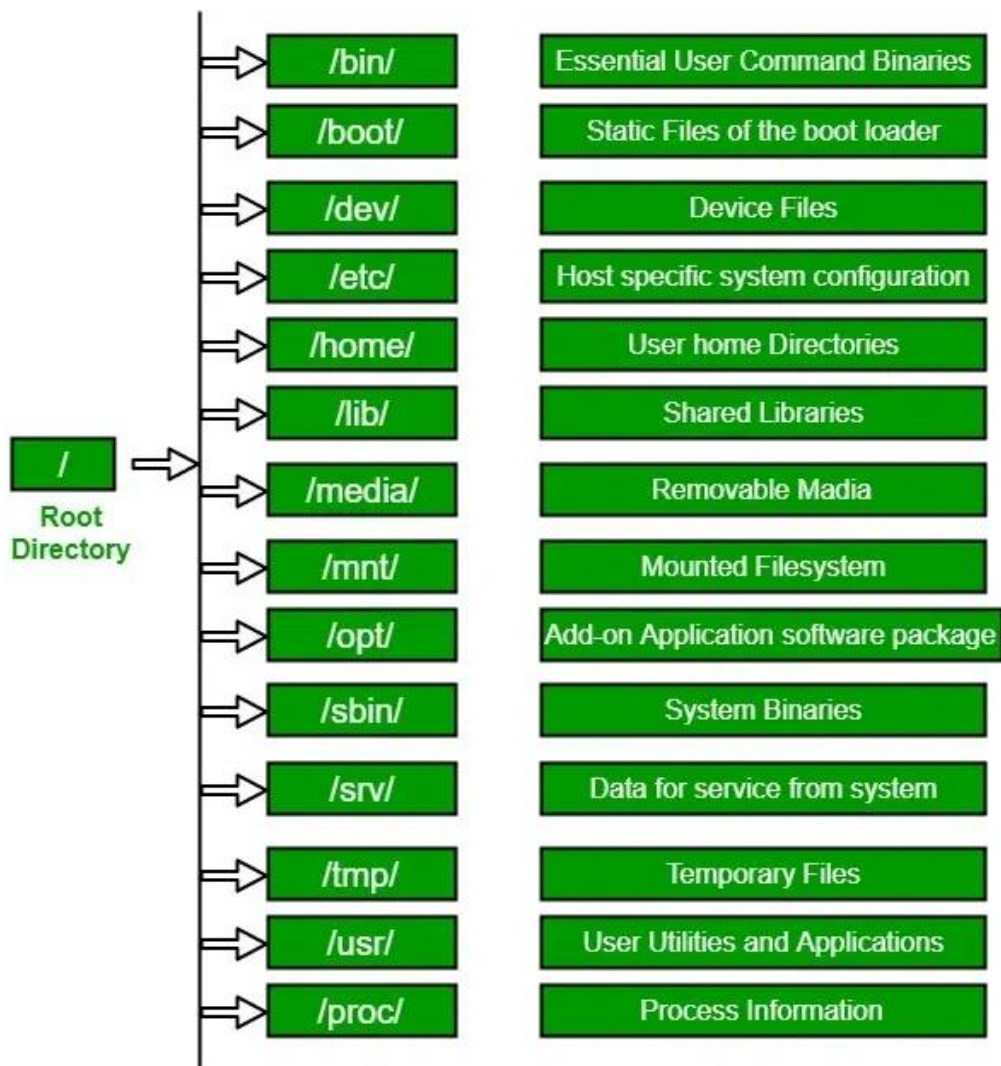
data storage in Linux/Unix operating systems

The Linux/Unix file system hierarchy base begins at the root and everything starts with the root directory.

Linux File Hierarchy Structure

The Linux File Hierarchy Structure or the Filesystem Hierarchy Standard (FHS) defines the directory structure and directory contents in Unix-like operating systems. It is maintained by the Linux Foundation.

- In the FHS, all files and directories appear under the **root directory /**, even if they are stored on different physical or virtual devices.
- Most of these directories exist in all UNIX operating systems and are generally used in much the same way; however, the descriptions here are those used specifically for the FHS and are not considered authoritative for platforms other than Linux.



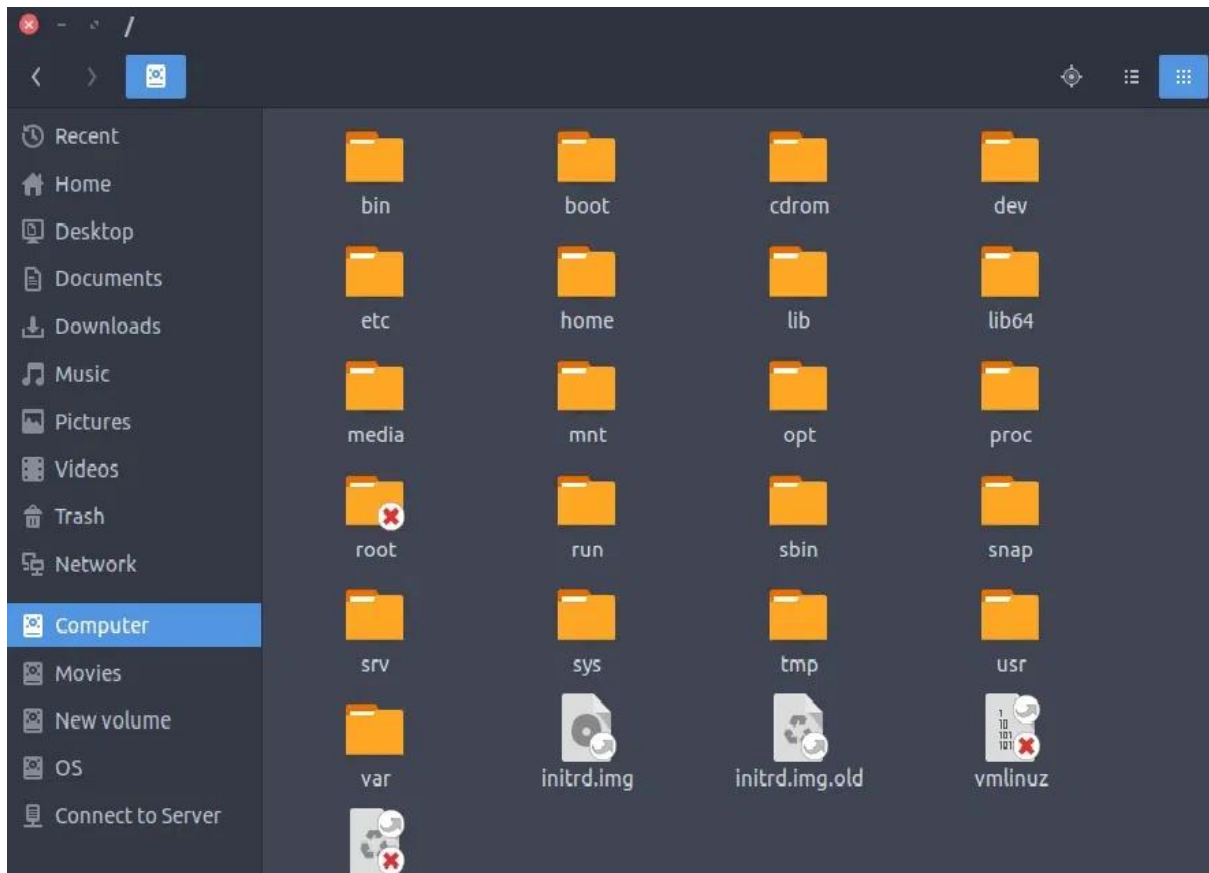
linux-directory

1. / (Root):

At the top of every Linux file system is the **root directory** represented by a forward slash /. It's the base point, and no directory exists above it. If you look at the file system graphically, you'll see all other directories branching from this root directory.

- Every single file and directory **start from the root directory**.
- The only root user has the right to write under this directory.
- **/root is the root user's home directory, which is not the same as /**

Only the root user has permission to modify contents inside this directory. Regular users cannot make changes here. For example, if you attempt to create a file in / as a non-root user, you'll encounter a "Permission Denied" error.

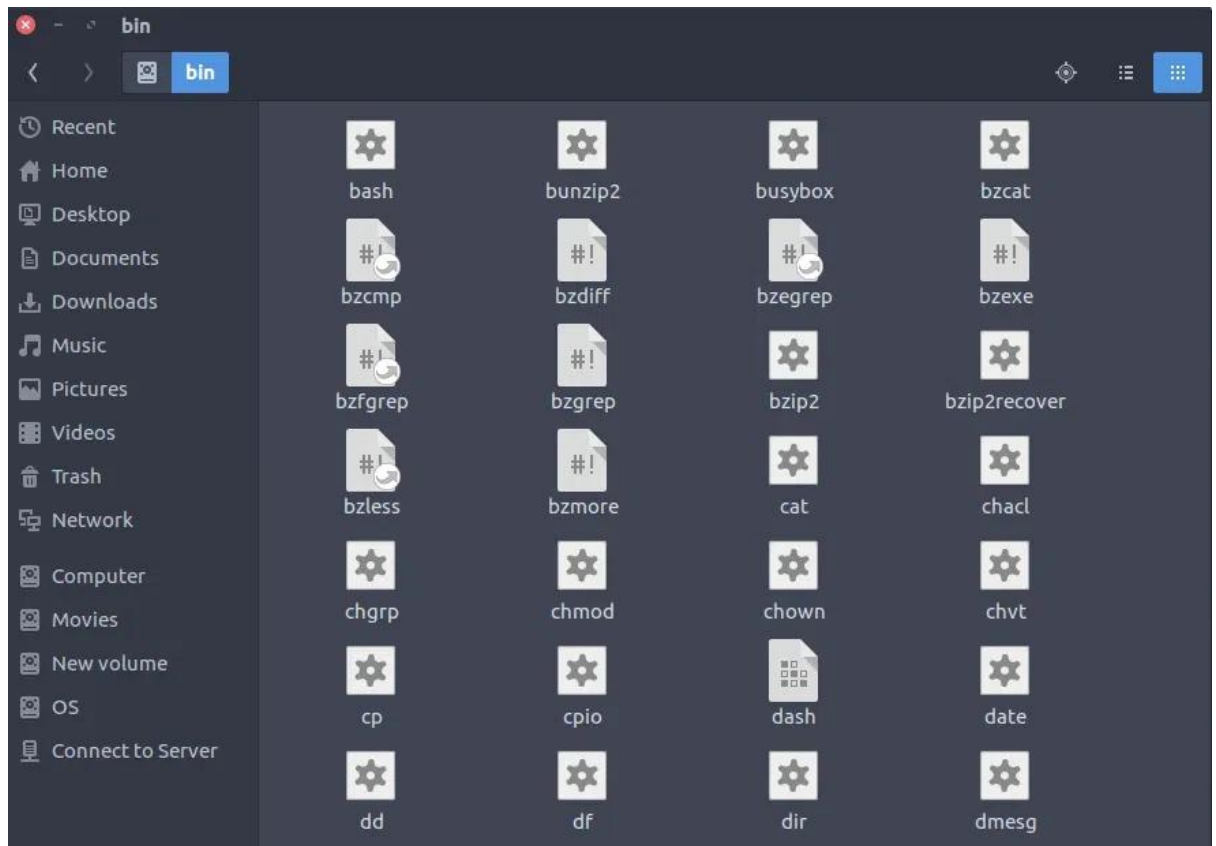


Root-linux-directory

2. /bin :

The /bin directory **contains essential commands and binaries needed by all users, including cp, ls, ssh, and kill**. These commands are universally available across user types.

- Contains binary executables.
- Common linux commands you need to use in single-user modes are located under this directory.
- Commands used by all the users of the system are located here e.g. ps, [ls](#), [ping](#), [grep](#), [cp](#)

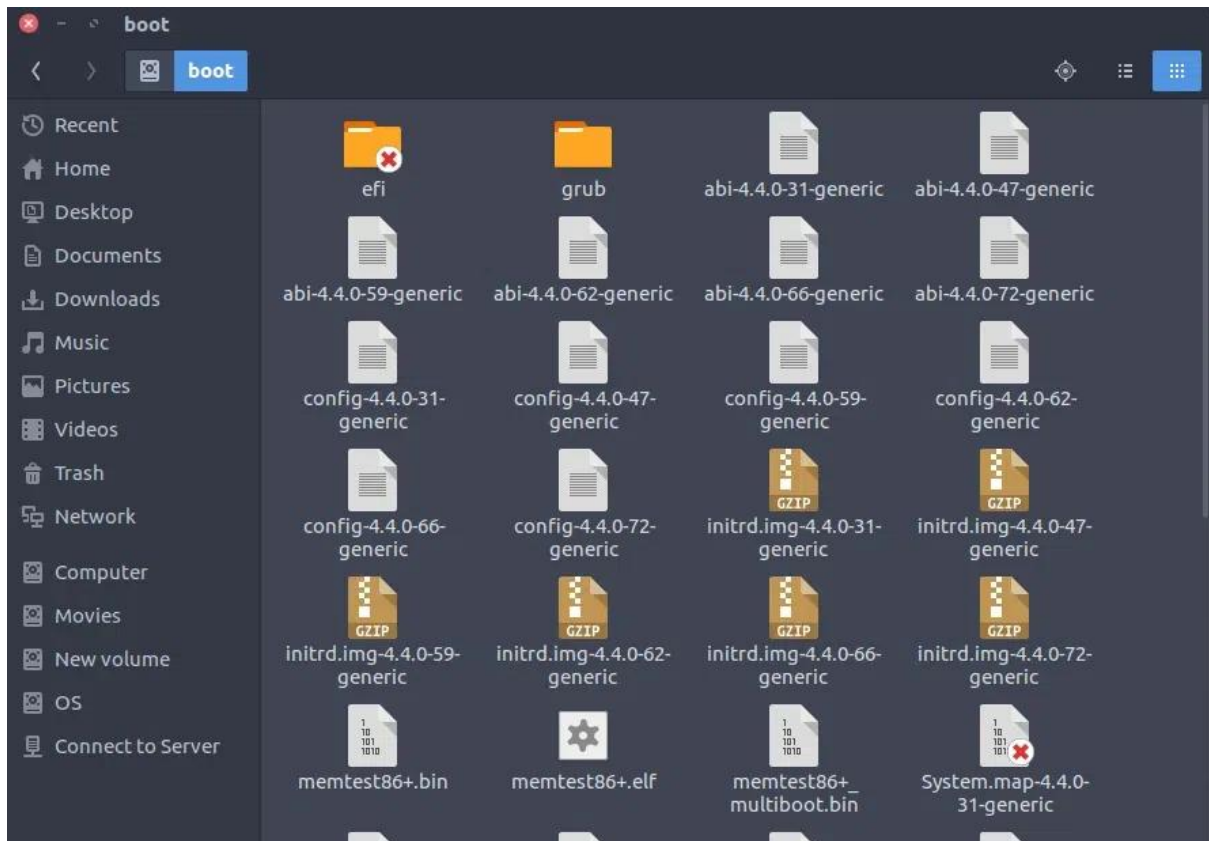


bin-linux-directory

3. /boot :

This directory stores **all files required for booting** the system. It includes the GRUB bootloader configuration and essential kernel files that are **loaded during startup**.

- Kernel **initrd**, **vmlinuz**, **grub** files are located under /boot
- Example: initrd.img-2.6.32-24-generic, vmlinuz-2.6.32-24-generic



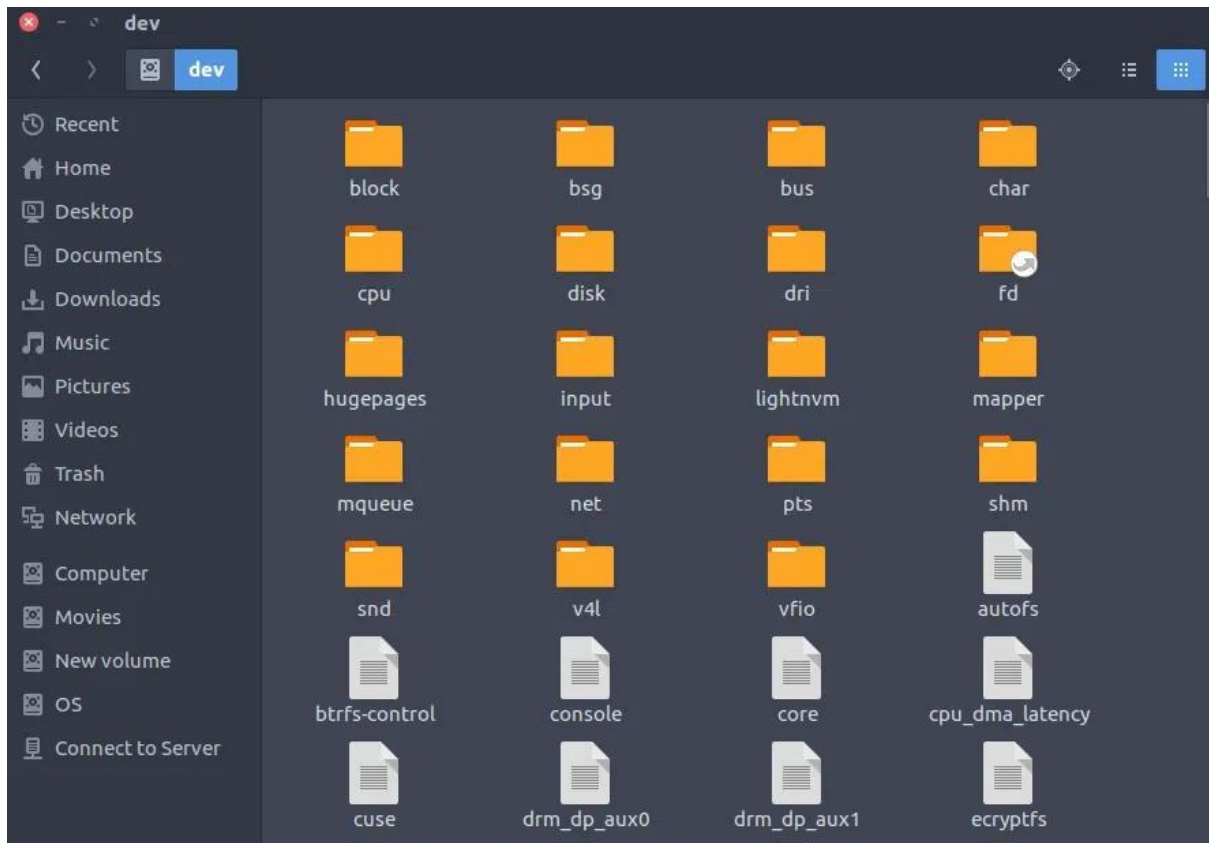
boot-linux-directory

4. /dev :

Device files in Linux are stored in the /dev directory. These are special files that act as interfaces between **hardware and software**. Device files are of two types: **block devices** (e.g., hard drives) and **character devices** (e.g., microphones and speakers).

Examples include /dev/sda1 for disk partitions.

- These include terminal devices, usb, or any device attached to the system.
- Example: /dev/tty1, /dev/usbmon0



dev-linux-directory

5. /etc :

Short for "**E**ditable **T**ext **C**onfiguration," /etc contains **configuration files for system applications, users, services, and tools** or it contains the Host-specific system-wide configuration files. For example, user details like UID and local addresses are defined here.

Physical file structure of Unix

A file system is a group of files and relevant information regarding them. Your whole hard disk may comprise a single file system or it may be partitioned into several file systems.

The disk space allotted to a Unix file system is made up of 'blocks' each of which are typically of 512 bytes. Some file systems have blocks of 1024 or 2048 bytes as well.

All the blocks belonging to the file system are logically divided into four parts . 1) Boot Block, 2) Super Block, 3) I-node Block, 4) Data Block.

Boot Block	Super Block	I-node Block	Data Block
------------	-------------	--------------	------------

- **Boot Block:**

Boot Block or Block 0 is the first block of the file system. This block is normally reserved for booting procedures. It is also called Master Boot Record (MBR). The boot block contains the partition table and a small 'bootstrapping' program. When the system is booted, the system BIOS checks for the existence of the first block of hard disk and loads the entire segment (code) of the boot block into memory. It then hands over control to the bootstrapping program. This in turn loads the kernel (the file unix) into memory. However, the bootstrapping program is read in form the boot block of the root(main or track zero) file system. For other file system, this block is simply kept blank

- **Super Block:**

The Super Block or Block 1 is the 'balance sheet' of every Unix file System. It contains global file information about disk usage and availability of the data block and I-nodes. It contains

- The Size of the file system.
- The length of the file system's Logical block
- Last time of Updating
- The number of free data blocks available and a partial list of immediately allocable free data blocks.
- Number of free I-nodes available and a partial list of immediately usable I-nodes.
- The state of the file System (whenever 'clear' or 'dirty')

As with I-nodes, the kernel also maintains a copy of the Super Block in memory.

- **I-node Block:**

We know that all entities in UNIX are treated as files. I-nodes are used to describe the file characteristic and the locations of the data blocks which store the contents of the system. The information related to all these files (not the contents) is stored in an I-node table on the disk. For each file, there is an I-node entry in the table. Each entry is made up of 64 bytes and contains the relevant details for that file. These details are:

- File Type (regular, directory, device etc)
- Number of links (the number of aliases the file has)
- The numeric UID of the owner.
- The numeric GUID of the owner.
- File mode(file access permission)
- Size of the file
- Date and Time of last modification of file data
- Date and Time of last access of file data
- Date and Time of last change of the I-node

- Addresses of blocks where the file is physically present

The I-node is accessed by number called *I-node Number*. This number is unique for every file in a single file system.

- **Data Block:**

The remaining space of the file system is taken up by Data block or Storage block. For a directory each data block contains 16 bytes entries. Each such entry would have a file or subdirectory name up to 14 bytes and 2 bytes would be taken for the I-node. Thus, for directories, the table of filenames and their I-nodes is available.

e.g. (directory called /usr)

14 bytes	2 bytes
File name 1	I-node 1
File name 2	I-node 2
File name 3	I-node 3

The operation system's files, all data and programs created by users reside in this area.